

Project 3 (in Java): Linked-list implementation of Stack and Queue. Your program will perform the following tasks:

- A) Build a stack from the data in inFile:
- 1) create a linked-list stack with a dummy node, where top points to the dummy node;
 - 2) read one data at the time from the input file, inFile, and print the data to outFile1;
 - 3) create a new node with data;
 - 4) push newNode on top of the stack (after the dummy node);
 - 5) repeat step 2 to 4 while inFile is not empty.
 - 6) print linked-list stack to outFile1.
- B) Build a queue from nodes in the stack:
- 1) create a linked list queue where head and tail point to a dummy node;
 - 2) pop the node from the stack and print the data in the node to outFile2;
 - 3) insert the popped node at the back of the queue.
 - 4) repeat step 2 to 3 while stack is not empty.
 - 5) print linked-list queue to outFile2.

What you need to do:

1. Implement and debug your program with respect to the specs given below until your program passes compilation.
2. You will be given two data files: data1 and data2; data1 contains only a few integers and data2 contains more integers.
3. Run your program using data1 and eyeball the output of your program for correctness.
4. When your program produces correct output from data1, then run your program using data2.

=====

*** Include in your hard copy *DOC.docx file as follows:

- Cover page
- Source code
- print data1 // with caption *** below is the input data1 ***
- print outFile1 from data1 // with caption *** The outFile1 from data1 ***
- print outFile2 from data1 // with caption *** The outFile2 from data1 ***
- print logFile from data1 // with caption *** The logFile from data1 *** // limited to 3 pages, if more.
- inFile from data2 // with caption *** below is the input data2 ***
- outFile1 from data2 // with caption *** The outFile1 from data2 ***
- outFile2 from data2 // with caption *** The outFile2 from data2 ***
- logFile from data2 // with caption *** The logFile from data2 *** // limited to 3 pages, if more.

Note: You must use args to open input file and 4 output files.

-3 points deduction if you hard-code your file names!

Language: Java

Project name: Linked-list implementation of Stack and Queue

Project points: 10 pts

Due Date: Soft copy (*.zip) and hard copies (*.docx):

(10/10 pts): on time, 9/26/2026, Saturday before midnight (11:59pm)

(-10/10 pts): non-submission, 9/26/2026, Saturday after midnight

*** Name your soft copy and hard copy files using the naming convention given in Project Submission Requirements.

*** All on-line submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as below; otherwise, your submission will be rejected.

Email subject: (313) first name last name <Project 3: Linked-list implementation of Stack and Queue (Java)>

*** Inside the email body includes:

- Your answer to the 5 questions given in your email body. (-1 if not include.)
- Screen recoding link. (-1 if not submit screen recording).

I. Input:

1) inFile (use args [0]): a text file contains a set of positive integers, not in any particular format.

II. Outputs:

1) outFile1 (use args [1]): for stack outputs.

2) outFile2 (use args [2]): for queue outputs.

3) logFile (use args [3]): as spec dictates

III. Data structure:

- listNode class // as public

- (int) data

- (listNode) next

Methods:

- constructor (data) //Assign listNode' data with given data and assign listNode's next null, ie.

//this->data = data; this->next = nullptr;

- printNode (node) // print in the format as below:

(node's data, node's next's data) →

For example:

(8, 11) →

- LLStack class

- (listNode) top

Methods:

- constructor (...) // do as you need.

- push (node) // insert newNode after dummy node; i.e.,

// node.next ← top.next

// top.next ← node

- (bool) isEmpty () // if top's next is null returns true, otherwise returns false.

- listNode pop () // if stack is not empty, removes and returns the node on top of the stack

// otherwise, print "stack is empty" to outfile1.

// Use code given in class.

- buildStack (inFile) // build a stack from the data in inFile. **Algorithm is given below.**

- printStack (...) **// On your own.**

//the method calls printNode(...) to print all nodes in the list to outFile3, in the following format:

Top → (-999, next's data₁) → (data₁, next's data₂) → → (data_j, NULL) → NULL

For example:

Top → (-999, 8) → (8, 11) → (11, 21) → → (54, 87) → (87, NULL) → NULL

- LLQueue class

- (listNode) head // head always points to the dummy node!

- (listNode) tail // tail always points to the last node of the queue.

Methods:

- constructor(...) // as you need.

- insertQ (...) // insert the newNode after the tail of Q; **use code given in class.**

- (bool) isEmpty (...) // Returns true if head's next is null; returns false otherwise.

- buildQueue (...) // build a queue from nodes in the stack. **Algorithm is given below.**

- printQueue (...) **// On your own.** the method calls printNode(...) to print all nodes in the list to outFile2,

//in the following format:

Head → (-999, next's data₁) → (data₁, next's data₂) → → (data_j, NULL) ← Tail

For example:

Head → (-999, 8) → (8, 11) → (11, 21) → → (54, 87) → (87, NULL) ← Tail

IV. main (...) // Study Basic C++ and Java I/O posted.

Step 0: check args length is 4 and check all files, one-by-one, it each can be opened.

inFile ← open input file from args [0] // Define inFile as Scanner, see Project0A.

outFile1, outFile2, logFile ← open from args [1], args [2], args [3]

Step 1: S ← define S as a LLStack, i.e.,

// LLStack S;

// S.top ← new listNode (-999)

Step 2: logFile ← “***calling buildStack ()”

S.buildStack (inFile, logFile)

outFile1 ← “*** Printing the Stack ***”

S.printStack (outFile1)

Step 3: Q ← define Q as a LLQueue, i.e.,

// LLQueue Q;

// Q.head ← new listNode (-999);

// Q.tail ← Q.head;

Step 4: logFile ← “*** calling buildQueue ()”

Q.buildQueue (S, outFile2, logFile)

outFile1 ← “*** Printing the Queue***”

Q.printQueue (outFile2)

Step 4: close all files

V. buildStack (inFile, logFile)

Step 0: logFile ← write “*** Entering buildStack () !”

Step 1: data ← read a data from inFile // Use scanner

logFile ← “***input data is” // write data one data per text line

Step 2: newNode ← creates a listNode for data, using “new” as taught in class.

Step 3: push (newNode)

Step 4: repeat step 1 to step 3 until inFile is empty.

// From Step 1 to Step 4.

// Use while (inFile.hasNextInt())

// and use

// data ← inFile.nextInt() to read the data from inFile

Step 5: logFile ← write “*** Leaving buildStack ()!”

VI. buildQueue (S, outFile2, logFile)

Step 0: logFile ← write “*** Entering buildQueue ()!”

Step 1: newNode ← S.pop ()

logFile ← “*** After pop stack, newNode’s data is” //write newNode’s data; one data per text line.

Step 2: outFile2 ← “***After pop stack, newNode’s data is” //write newNode’s data; one data per text line.

Step 3: insertQ (newNode)

Step 4: repeat step 1 to step 3 until S is empty.

Step 5: logFile ← write “***leaving buildQueue ()!”

IV. (listNode*) deleteQ (Q, outFile2, logFile)

Step 0: logFile $\leftarrow$ write “entering deleteQ ()”

Step 1: if isEmpty (Q) // Q’s tail == Q’s head

 outFile2 $\leftarrow$ “Q is empty”

 logFile $\leftarrow$ “Q is empty”

 return null

Step 2: (listNode*) temp $\leftarrow$ Q.head’s next

Step 3: if tail == temp

 tail $\leftarrow$ head

Step 4: Q.head’s next $\leftarrow$ temp’s next

Step 5: temp’s next $\leftarrow$ null

Step 6: logFile $\leftarrow$ write temp’s data

 logFile $\leftarrow$ write “leaving deleteQ ()”

Step 7: return temp